

Production-Grade RAG Pipeline

A 4-Week End-to-End Implementation, Evaluation & Deployment Blueprint (Cursor IDE, FAISS/Qdrant, FastAPI & GitHub Actions)

EXECUTIVE SUMMARY: This implementation plan guides you from a basic vector search script to a production-ready Retrieval-Augmented Generation (RAG) system built directly with FastAPI, FAISS/Qdrant, and OpenAI/Cohere free-tier APIs. It eliminates restrictive framework abstractions (like LangChain) in favor of explicit, testable python modules, incorporating Reciprocal Rank Fusion (RRF) hybrid search, cross-encoder reranking, SSE token streaming, async document ingestion, and LLM-as-Judge automated evaluation gates.

Architectural & Technology Stack

Layer / Module	Technology Choices	Key Purpose & Design Rationale
IDE & Workspace	Cursor IDE + Python 3.11+	AI-accelerated refactoring, context-aware linting, and modular Python package layout.
Vector Database	FAISS (Local) / Qdrant (In-Memory)	Zero-process local iteration (FAISS) or fast payload-filtered vector store (Qdrant).
Sparse Search	BM25 (rank-bm25)	Keyword lexical matching fused with vector search using Reciprocal Rank Fusion (RRF).
Reranker & LLM	Cohere Rerank API + OpenAI gpt-4o-mini	Cross-encoder re-ordering for top chunks & cost-effective grounded answer generation.
API & Server	FastAPI + Uvicorn + SSE	Asynchronous HTTP endpoints, background job queueing, and Server-Sent Events token streaming.
Evaluation & CI	LLM-as-Judge + GitHub Actions	Automated RAGAS-style Faithfulness, Precision & Recall gates failing PRs below threshold.
Deployment	Docker Multi-Stage + Render / Railway	Containerized deployment with non-root security and GitHub Container Registry (GHCR) publishing.

4-Week Detailed Implementation Roadmap

Follow this sequential checklist to transition from isolated prototype scripts to an enterprise-grade deployed application.

Week 1: Core Ingestion & Hybrid Retrieval Engine

- Repository & Workspace Setup:** Initialize clean module directories (src/ingestion, src/retrieval, src/generation, src/api) and configure .env with OpenAI & Cohere API keys.

- **Document Ingestion & Chunking:** Build robust document loaders (`loader.py`) for PDF, Markdown, and TXT files. Implement `RecursiveCharacterTextSplitter` with 800-char chunk size and 150-char overlap.
- **FAISS / Qdrant Integration:** Generate dense vector embeddings (`SentenceTransformer` or `Cohere/OpenAI`) and index them into local FAISS or in-memory Qdrant.
- **BM25 + RRF Hybrid Retrieval:** Build sparse lexical search (`rank-bm25`) and merge dense vector results with sparse keyword scores using Reciprocal Rank Fusion (`RAG_HYBRID_ALPHA = 0.6`).
- **Cohere Cross-Encoder Reranking:** Re-order the top-20 retrieved candidates using Cohere's Rerank API (`reranker.py`) down to the top-5 most relevant context snippets.

Week 2: FastAPI Backend, SSE Streaming & Async Ingestion

- **FastAPI Application Framework:** Create `src/api/app.py` with liveness (`/healthz`), readiness (`/readyz`), and pipeline metrics (`/stats`) endpoints.
- **Synchronous & Streaming Query Endpoints:** Implement `POST /query` returning answers with precise source citations. Add `POST /query/stream` using Server-Sent Events (SSE) for real-time token streaming.
- **Async Document Ingestion Queue:** Implement `POST /ingest/async` returning a `202 Accepted` with a `job_id` to prevent HTTP timeout issues during large file parsing.
- **Context Budget Management:** Implement prompt context trimming (`max_context_chars`) to prevent mid-sentence token clipping and enforce system grounding prompts.

Week 3: LLM-as-Judge Evaluation & Observability

- **LLM-as-Judge Evaluation Suite:** Create an automated test runner (`scripts/evaluate.py`) evaluating `gpt-4o-mini` outputs against a golden JSONL dataset.
- **Four Core RAG Metrics:** Measure *Faithfulness* (groundedness), *Answer Relevance*, *Context Precision* (Average Precision @ k), and *Context Recall*.
- **Observability & Tracing:** Instrument Langfuse distributed tracing or OpenTelemetry (OTel) metrics for latency tracking, query counters, and token cost attribution.
- **Comprehensive Pytest Suite:** Write unit & integration tests (`pytest tests/`) covering RRF fusion math, chunking strategies, and API security guards.

Week 4: CI/CD Deployment, Docker & Blog Showcase

- **Docker Multi-Stage Build:** Write a production Dockerfile (`deps` → `model-cache` → `runtime`) enforcing non-root user execution and local file permission controls.
- **GitHub Actions CI/CD Pipeline:** Create `.github/workflows/ci.yml` for linting/tests and `evaluate.yml` for automated PR quality gate evaluation (`--fail-on-threshold`).
- **Cloud Deployment:** Deploy the application container to Render, Railway, or Hugging Face Spaces with environment variable secret management.
- **Technical Blog Post & Showcase:** Write a technical deep-dive post detailing your direct SDK architecture, hybrid retrieval gains, and LLM-as-Judge evaluation results.

Architectural Rationale: Direct SDKs vs. Framework Abstractions

By writing retrieval, fusion, reranking, and generation directly against vector databases, rank-bm25, and official LLM provider SDKs (rather than heavy framework abstractions), your application gains key engineering advantages: (1) **Explicit & Testable Logic:** RRF scoring and context trimming are plain Python functions covered by unit tests; (2) **No Versioning Surprises:** Eliminates dependency conflicts between framework versions and provider SDKs; (3) **Direct Stack Debuggability:** Tracebacks point directly to your ingestion or retrieval code rather than deep internal chains.

FastAPI Core Service Endpoints

HTTP Method	Endpoint Path	Function & Expected Behavior
GET	/healthz	Liveness probe returning status: ok for container orchestration.
GET	/stats	Pipeline metrics: chunk counts, active vector store, and embedding cache stats.
POST	/ingest/async	Enqueue asynchronous ingestion job (returns 202 Accepted + job_id).
GET	/ingest/jobs/{job_id}	Poll status and progress of a background ingestion task.
POST	/query	Synchronous Q&A; returning generated answer + structured chunk citations.
POST	/query/stream	Server-Sent Events (SSE) endpoint streaming response tokens in real-time.